UNIVERSIDAD NACIONAL EXPERIMENTAL DE TÁCHIRA
VICERRECTORADO ACADÉMICO
DECANATO DE DOCENCIA
DEPARTAMENTO DE INGENIERÍA INFORMÁTICA
REDES NEURONALES

Redes del tipo Kolmogorov-Arnold (KAN)

Autor:

Juan Diego Paredes Gámez V-31.357.791

Sección: 01

San Cristóbal, marzo de 2026

1. Introducción y Objetivos

La presente actividad tiene por objetivo complementar los conceptos estudiados sobre arquitecturas neuronales alternativas. Específicamente, se busca implementar y desarrollar una red del tipo Kolmogorov-Arnold (KAN) utilizando la librería pykan de Python. Una vez implementada la red, el objetivo central es documentar y comentar los resultados obtenidos a nivel de: ventajas y desventajas de la arquitectura, tiempo de entrenamiento y robustez del modelo.

2. Instrucciones para la ejecución del código

Para replicar los resultados de esta evaluación, es necesario seguir los siguientes pasos:

- Contar con un entorno de Python 3.x.
- Instalar las dependencias principales y secundarias requeridas por la librería KAN ejecutando el siguiente comando en la terminal:
    - ***pip install torch pykan scikit-learn sympy tqdm pyyaml matplotlib***
- Guardar el código fuente suministrado en los anexos en un archivo nombrado ***kan_modelo.py.***
- Ejecutar el script mediante el comando:
    - ***python kan_modelo.py***

3. Resultados de la Ejecución

Al ejecutar el modelo KAN sobre un conjunto de datos sintético para resolver un problema de regresión de la función **f(x, y) = exp(sin(pi*x) + y^2)**, la consola arrojó una salida indicando el progreso del entrenamiento, la creación de checkpoints del modelo y las métricas finales de desempeño.

```
--- Iniciando preparación de datos ---
--- Construyendo la red Kolmogorov-Arnold (KAN) ---
checkpoint directory created: ./model
saving model version 0.0
--- Iniciando entrenamiento ---
| train_loss: 4.87e-03 | test_loss: 5.28e-03 | reg: 0.00e+00 | : 100%|| 20/20 [00:04<00:00,  4.33it
saving model version 0.1

========================================
RESUMEN DE RESULTADOS KAN
========================================
Tiempo de entrenamiento: 4.6397 segundos
Pérdida en entrenamiento (RMSE): 0.002536
Pérdida en prueba (RMSE): 0.002717
```

Que se traduce en:

Resumen de Resultados KAN

- Tiempo de entrenamiento: 4.6397 segundos
- Pérdida en entrenamiento (RMSE): 0.002536
- Pérdida en prueba (RMSE): 0.002717

4. Análisis de la Red KAN

- **Ventajas y Desventajas de la arquitectura:**

Ventajas: Una de las principales fortalezas de las redes KAN es su interpretabilidad matemática. Al ubicar funciones de activación (B-splines) en las conexiones entre nodos en lugar de en los nodos mismos, la red puede aproximar funciones complejas utilizando una menor cantidad de parámetros en comparación con arquitecturas tradicionales como los Perceptrones Multicapa (MLP).

Desventajas: La principal limitación es su mayor costo computacional por parámetro. Evaluar funciones B-spline en cada conexión requiere operaciones más complejas que las multiplicaciones matriciales simples utilizadas en redes neuronales tradicionales, lo que puede dificultar su paralelización eficiente en hardware como GPUs.

Tiempo de entrenamiento: Para la arquitectura utilizada (2 entradas, 5 nodos ocultos y 1 nodo de salida) y 20 pasos de entrenamiento utilizando el optimizador LBFGS, el modelo registró un tiempo de 4.6397 segundos. Aunque el tiempo es muy reducido para este ejemplo sintético, teóricamente las redes KAN escalan de forma más lenta que los MLPs cuando se aplican a conjuntos de datos de gran tamaño.

Robustez del modelo: La métrica RMSE obtenida evidencia una buena capacidad de generalización. La diferencia entre la pérdida de entrenamiento (0.002536) y la pérdida de prueba (0.002717) es mínima, lo que sugiere ausencia de sobreajuste significativo. Además, gracias al control local proporcionado por las activaciones B-spline, las redes KAN presentan una robustez teórica interesante frente al problema del olvido catastrófico cuando se incorporan nuevos datos.

5. Conclusión

La implementación de la red neuronal de tipo Kolmogorov-Arnold (KAN) mediante la librería pykan se completó de manera exitosa, logrando resolver el problema de regresión sintética con una precisión excelente. Se comprobó empíricamente la rapidez de convergencia del modelo y su alta robustez frente al sobreajuste, reflejada en la mínima diferencia entre la pérdida de entrenamiento y la de prueba. Aunque las redes KAN exigen un mayor esfuerzo computacional por parámetro al evaluar funciones B-splines en sus conexiones, su alta interpretabilidad matemática y la capacidad de modelar patrones complejos utilizando menos parámetros las convierten en una alternativa sumamente poderosa frente a los Perceptrones Multicapa (MLP) tradicionales de caja negra.

6. Anexos: Código Fuente (kan_modelo.py)

```
import torch

from kan import KAN, create_dataset

import time




# 1. PREPARACIÓN DE DATOS



def preparar_datos():

    """

       Crea un conjunto de datos sintético para un problema de
regresión.

      La red intentará aprender la función subyacente: f(x, y) =
exp(sin(pi*x) + y^2)

    """

    # Función matemática objetivo a modelar

        f = lambda x: torch.exp(torch.sin(torch.pi*x[:,[0]]) +
x[:,[1]]**2)



    # create_dataset genera los tensores de entrenamiento y prueba

    dataset = create_dataset(f, n_var=2)

    return dataset




# 2. CONSTRUCCIÓN Y ENTRENAMIENTO DEL MODELO KAN
```

```python
def entrenar_evaluar_kan():

    print("--- Iniciando preparación de datos ---")

    dataset = preparar_datos()



    # Crear modelo KAN:



    print("--- Construyendo la red Kolmogorov-Arnold (KAN) ---")

    model = KAN(width=[2, 5, 1], grid=5, k=3, seed=0)



    print("--- Iniciando entrenamiento ---")

    start_time = time.time()



     # Entrenar el modelo usando el optimizador LBFGS (recomendado
para KANs)

    results = model.fit(dataset, opt="LBFGS", steps=20, log=10)



    end_time = time.time()

    tiempo_entrenamiento = end_time - start_time



    # Extraer resultados

    train_loss = results['train_loss'][-1]

    test_loss = results['test_loss'][-1]



    # Imprimir resumen de la ejecución

    print(f"\n==========================================")
```

```
    print("RESUMEN DE RESULTADOS KAN")

    print("=========================================")

       print(f"Tiempo de entrenamiento: {tiempo_entrenamiento:.4f}
segundos")

    print(f"Pérdida en entrenamiento (RMSE): {train_loss:.6f}")

    print(f"Pérdida en prueba (RMSE): {test_loss:.6f}")



if __name__ == "__main__":

    entrenar_evaluar_kan()
```